

Matrices: In a matrix, rows are the ones that run horizontally and columns are the ones that run vertically. In R programming, matrices are two-dimensional, homogeneous data structures.

Creating a matrices: To create a matrix in R we need to use the function called `matrix()`.

Syntax: `m<-matrix(c(data), nrow, ncol,byrow,dimnames=list(row_name,col_name))`

- **data** is the input vector which becomes the data elements of the matrix.
- **nrow** is the number of rows to be created.
- **ncol** is the number of columns to be created.
- **byrow** is a logical clue. If TRUE then the input vector elements are arranged by row.
- **dimname** is the names assigned to the rows and columns.

Ex: `m<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,ncol=3)`

```
print(m)
```

Add new rows and columns: We can use `cbind` and `rbind` to add columns and rows.

Syntax: `cbind(matrix_name,vector_list)`

```
rbind(matrix_name,vector_list)
```

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,ncol=3)
```

```
print(cbind(m,c(10,11,12)))
```

```
print(rbind(m,c(13,14,15)))
```

Remove rows and columns:

Syntax: `matrix_name<-matrix_name[-row,-col]`

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,ncol=3)
```

```
m<-m[-1,-3]
```

```
print(m)
```

Reshaping the matrix (Transpose):

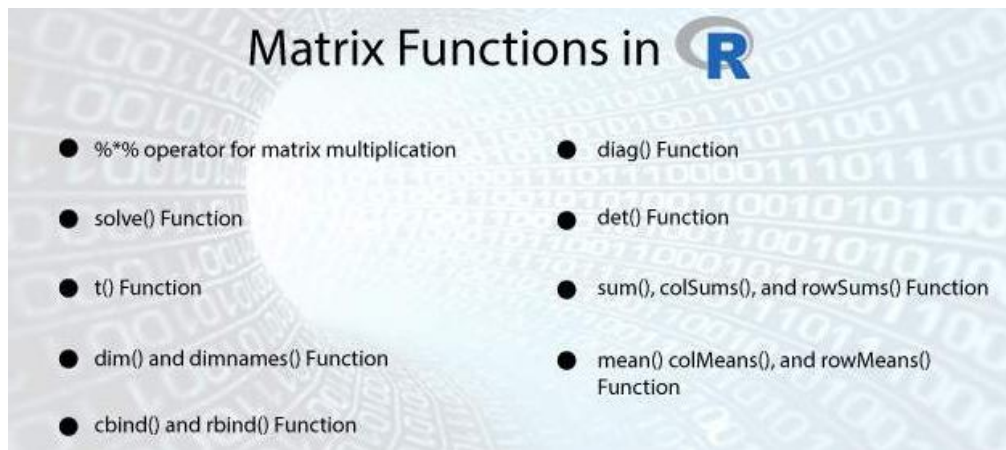
Syntax: `matrix_name<-t(matrix_name)`

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,ncol=3)
```

```
m<-t(m)
```

```
print(m)
```

Operation on matrix:

1.solve(): The `solve()` function in R programming takes a matrix as an argument and then returns the inverse of that matrix.

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-solve(m)
```

```
print(m)
```

2.diag(): In R, when you are in a situation where you need to create a diagonal matrix, you can use the `diag()` function that allows you to create a diagonal matrix. This function also helps you in extracting the diagonal of an existing matrix which already has been created.

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-diag(m)
```

```
print(m)
```

3.det(): The `det()` function in R allows us to find out the determinant of a matrix.

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-det(m)
```

```
print(m)
```

4.sum(), rowSums(), colSums(): The sum() function computes the sum of all elements from the matrix. The colSums() function allows us to find out the column-wise sum of elements for the matrix. The rowSums() function allows us to find out the row-wise sum of the elements for the given matrix.

Ex:

```
i)m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-sum(m)
```

```
print(m)
```

```
ii)m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-rowSums(m)
```

```
print(m)
```

```
iii)m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-colSums(m)
```

```
print(m)
```

5.mean(), colMeans(), rowMeans(): Allows us to find out the mean value for all elements, column-wise mean values, and row-wise mean values for a given matrix in R.

Ex:

```
i)m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-mean(m)
```

```
print(m)
```

```
ii)m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-rowMeans(m)
```

```
print(m)
```

```
iii)m<-matrix(c(1,2,3,4,5,6,7,12,34),nrow=3,ncol=3)
```

```
m<-colMeans(m)
```

```
print(m)
```

6.%% operator for Matrix Multiplication: The multiplication for matrices works in a different way. If you want to multiply two matrices, you need to use the %% operator. This operator does the matrix multiplication in a proper way.

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,ncol=3)
n<-matrix(c(10,11,12,13,14,15,16,17,18),nrow=3,ncol=3)
r<-m%*%n
print(r)
```

7.upper.tri(): Is used to return a matrix of logical values with upper triangle as TRUE.

Syntax: upper.tri(x, diag)

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,ncol=3)
print(upper.tri(m,diag=TRUE))
```

8.lower.tri(): Is used to return a matrix of logical values with lower triangle as TRUE.

Syntax: lower.tri(x, diag)

Ex:

```
m<-matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,ncol=3)
print(lower.tri(m,diag=TRUE))
```

Lists: A list in R programming is a object consisting of an ordered collection of objects. Lists are one-dimensional, heterogeneous data structures. The list can be a list of vectors, a list of matrices, a list of characters, a list of functions, and so on. A list is a vector but with heterogeneous data elements. A list in R is created with the use of the list() function. R allows accessing elements of an R list with the use of the index value. In R, the indexing of a list starts with 1 instead of 0.

Creating a list: A list in R is created with the use of the list() function.

Ex:

```
my_list<- list(name = "Sudheer", age = 25, city = "Delhi")
print(my_list)
```

General list operation:

1.length(): This function is used to return the length of every element in a list.

Ex:

```
my.list <- list(name = c("A", "B", "C"), age = c(20, 30, 40, 50))  
print(lengths(my.list))
```

2.names(): The names() function in R is used to get or set the name of an object. The object can be a list, vector, matrix, Data frame.

Ex:

```
a <- list('Spark','R','Java','Python')  
names(a) <- c('AA','AB','AC','AD')  
print(a)
```

3.[[]]: returns a object of the class of item contained in the list.

Ex:

```
my_list<-list(std=c(5,6,7,8,9,10),picnic="Water Kingdom")  
a = [[picnic]]  
print(a)
```

4.c(): Two R lists can be concatenated using the concatenation function.

Ex:

```
l<-list(1,2,3)  
n<-list("Apple","Banana","Cherry")  
new<-c(l,n)  
print(new)
```

5.c(list_name,index no./value): To add an item to the list, we can use this function.

Ex:

```
my_numbers = c(1,5,6,3)  
print(c(my_numbers,45))
```

6.unlist(): unlist() function in R language is used to convert a list to vector. It simplifies to produce a vector by preserving all components.

Ex:

```
my_list <- list(m1 = c(1,2),m2 = c(3,4),m3 = c(5,6))  
print(unlist(my_list))
```

Special function:

1.lapply(): lapply is a function which is apply to each element of the list.

Ex:

```
m<-list(12,344,55,56,67,8)
```

```
print(lapply(m,sqrt))
```

2.sapply(): sapply is a function which is apply to each element of the list and simplifies the output.

Ex:

```
m<-list(12,344,55,56,67,8)
```

```
print(sapply(m,sqrt))
```

3.vapply(): vapply is a function which is apply to each element of the list and return a vector.

Ex:

4.mapply(): mapply is a function which is apply to each element of multiple list.

Syntax: mapply(func, list1, list2, ...)

Ex:

```
A = list(c(1, 2, 3, 4))
```

```
B = list(c(2, 5, 1, 6))
```

```
result = mapply(sum, A, B)
```

```
print(result)
```

Recursive list: A list can contain other list has element. Recursive function can be used to diverse and manipulate recursive list.

Ex:

```
my<-list(1,list(2,3),4,5,6)
```

```
print(my[[2]])
```

Visualization in R: Data visualization is a technique used for the graphical representation of data. By using elements like scatter plots, charts, graphs, histograms, maps, etc., we make our data more understandable. Data visualization makes it easy to recognize patterns, trends, and exceptions in our data.

Basic Visualization:

Functions use to perform basic visualization:

1.Scatter plots [plot(x,y)]: A scatter plot is composed of many points on a Cartesian plane. Each point denotes the value taken by two parameters and helps us easily identify the relationship between them.

2.Bar plots[barplot(x)]: A barplot is used to display the relationship between a numeric and a categorical variable. A bar chart represents data in rectangular bars with length of the bar proportional to the value of the variable.

4.Box plots[boxplots(x)]: The statistical summary of the given data is presented graphically using a boxplot.

5.Line plots[plot(x,type="l")]: A line graph has a line that connects all the points in a diagram.

Advanced Visualization:

1.Interactive:

2.Heatmaps: heatmap(), geom_point(), ggplot2.

3.Time series: ts.plot(), geom_line(), ggplot2.

4.Network Analysis: plot(), geom_edge, ggplot2.

Libraries: ggplot2, lattice, Plotly, Shiny, Network D3.

Feature	Basic Visualization	Advanced Visualization
Purpose	Simple data exploration	In-depth analysis and storytelling
Complexity	Low	High
Customization	Minimal	Extensive customization options
Interactivity	None	High (e.g., interactive dashboards)
Examples	Scatter plots, bar charts, histograms	Heatmaps, 3D plots, interactive visualizations
Libraries/Tools	ggplot2, base R plotting	ggplot2, plotly, Shiny, rgl

Creating 3D plots:

1.3D Scatter Plots: `scatterplot3d()`, `Plotly::Plot_ly(type="scatter3d")`.

2.3D Surface Plots: `Persp()`, `Plotly::Plot_ly(type="surface")`.

3.3D Bar Plots: `barplot3d()`, `Plotly::Plot_ly(type="bar3d")`.

Libraries: `Plotly`, `rgl`, `ScatterPlot3d`.

INPUT/OUTPUT: Input/output (I/O) is the technical term for reading and writing data: the process of getting information into a particular computer system (in this case R) and then exporting it to the 'outside world' again

1.Reading and writing in CSV files.

Ex:

```
data<-data.frame(Name=c("John","Mary","David"),Age=c(25,31,42))
```

```
write.csv(data,"input.csv",row.names=FALSE)
```

```
read_data<-read.csv("input.csv")
```

```
print(read_data)
```

```
write.csv(read_data,"output.csv",row.names=FALSE)
```

2.Reading and writing in Excel files.

Ex:

```
data<-data.frame(Name=c("John","Mary","David"),Age=c(25,31,42))
```

```
write.xlsx(data,"input.xlsx",sheetName="Sheet1")
```

```
read_data<-read.xlsx("input.xlsx",sheetIndex=1)
```

```
print(read_data)
```

```
write.xlsx(read_data,"output.xlsx",sheetName="Sheet1")
```

3.Reading and writing in Text files.

Ex:

```
data<-data.frame(Name=c("John","Mary","David"),Age=c(25,31,42))
```

```
write.table(data,"input.txt",row.names=FALSE)
```

```
read_data<-read.table("input.txt",header=TRUE)
```

```
print(read_data)
```

```
write.table(read_data,"output.txt",row.names=FALSE)
```

Common function in R:

1.read.csv(): Read CSV files.

2.read.xlsx(): Read Excel Files (requires xlsx package)

3.read.table(): Read text files.

4.write.csv(): Write in CSV files.

5.write.xlsx(): Write in Excel files.

6.write.table(): Write in Text files.

Data Frame: It is a heterogeneous type.

Creating Data frame: Use the data.frame() function to create a data frame.

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),stringsAsFactors=FALSE)  
  
print(emp_data)
```

Getting structure of R data frame:

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),stringsAsFactors=FALSE)  
  
str(emp_data)
```

Extracting data from data frame:

1.We can extract the specific col from a data frame by specifying the col name:

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),stringsAsFactors=FALSE)  
  
extract<-data.frame(emp_data$employee_id,emp_data$salary)  
  
print(extract)
```

2.We can extract the specific row from a data frame by specifying the row name:

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),stringsAsFactors=FALSE)  
  
extract<-emp_data[1,]  
  
print(extract)
```

3.We can extract the specific rows corresponding to specific col:

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
extract<-emp_data[c(2,3,c(1,4))]  
  
print(extract)
```

Adding row in data frame: By using rbind() we can add row in data frame.

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
x<-list(4,"Vivek",650,"23/12/20")  
  
print(rbind(emp_data,x))
```

Adding column in data frame: By using cbind() we can add column in data frame.

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
x<-c("Dentist","Engineer","Doctor")  
  
print(cbind(emp_data,Professions=x))
```

Delete column: By assigning "NULL" value we can delete the column.

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
emp_data$date<-NULL  
  
print(emp_data)
```

Delete row: [-n,] give row number with negative sign in the square bracket to remove the row.

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
emp_data<-emp_data[-1,]  
  
print(emp_data)
```

summary(): The summary function returns the minimum, maximum, mean, median, etc.

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
print(summary(emp_data))
```

Special Function in Data frame:

1.head(): Function which returns the first n rows of the data frame.

Syntax: head(x, n=number)

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
print(head(emp_data,2))
```

2.tail(): Function which returns the last n rows of the data frame.

Syntax: tail(x, n=number)

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
print(tail(emp_data,1))
```

3.info(): It gives data frame summary.

Ex:

4.describe(): It gives statistical summary.

Ex:

```
emp_data<-  
data.frame(employee_id=c(1:3),employee_name=c("John","Mary","David"),salary=c(500,600,550),date=c("21/10/24","22/2/23","03/4/22"),stringsAsFactors=FALSE)  
  
x<-describe(emp_data)  
  
print(x)
```

5.groupby(): It will take the group of the specify name.

Merging Data Frame:

Ex:

```
df1<-pd.dataframe({'Id':[1,2],'Name':['John','Argon']})  
df2<-pd.dataframe({'Id':[1,2],'Age':[26,89]})  
merged<-pd.merge(df1,df2,on="Id")  
print(merged)
```

Factors: A factor is a categorical variable that stores data as levels.

1.Create a factor:

Ex:

```
gender<-factor(c("Male","Female","Male"))  
print(gender)
```

2.Ordered factor:

Ex:

```
gender<-factor(c("small","medium","large"),levels=c("small","medium","large"),ordered=TRUE)
print(gender)
```

3.Un-ordered factor:

Ex:

```
gender<-factor(c("small","medium","large"),levels=c("small","medium","large"),ordered=FALSE)
print(gender)
```

1.Creating a table:

Ex:

```
td<-table(c("male","female","male"),(c("young","old","young")))
print(td)
```

2.Converting a table to a data frame:

Ex:

```
td<-table(c("male","female","male"),(c("young","old","young")))
df<-as.data.frame(td)
print(df)
```

3.Creating a frequency in table:

Ex:

```
list<-c("apple","mango","cherry","mango")
print(table(list))
```

4.Contingency table:

Ex:

```
gender<-c("male","female","male")
smoker<-c("yes","no","yes")
r<-table(gender,smoker)
print(r)
```

5.Table with statistics:

Ex:

```
age<-c(25,67,78)
gender<-c("male","female","male")
r<-summary(gender~age)
print(r)
```

Central tendency: It is a statistical measures that describe the middle value of the data set. It aims to provide a single value that best represents the entire data set.

1.mean(): This is the average value of data set.

2.median(): This is the middle value of data set when sorted in descending order.

3.summary(): This is the frequently occurring value in the data set. This is also known as Modes.

Ex:

```
data<-c(1,2,3,3,4,5,6,7,8,9,10)
r<-mean(data)
r<-median(data)
r<-summary(data)
print(paste(r))
```

Variability: It refers to the dispersion of data points within a data set or distribution. It measures how much individual point deviate from the central tendency.

1.Range: This is the difference between maximum and minimum value.

Ex:

```
data<-c(1,2,3,3,4,5,6,7,8,9,10)
r<-max(data)-min(data)
print(paste(r))
```

2.Variance: Average sphere difference between the mean.

Ex:

```
data<-c(1,2,3,3,4,5,6,7,8,9,10)
r<-var(data)
print(paste(r))
```

Handle missing Value:

is.na(): It returns Boolean value if NA values is present in vector.

Ex:

```
data<-c(1,2,3,3,4,5,6,NA,7,8,9,NA,10,NA)
```

```
r<-is.na(data)
```

```
print(r)
```

Outliers: They are data points that are significantly different from other data points in a data set.

1.Uni-variant outliers: Data points that are far away from the mean/median of a single variable.

2 Multi-variant outliers: Data points that are far away of multiple variable.

Visual method of outliers:

1.Detection of outliers: plot(), boxplot(), hist().

2.Handling outliers:

1.Removal [na.omit(),complete.cases()].

2.Transformation [log(),sqrt()]: Reduces the effect.

3.Imputation [mean(), median(), mode].

4.Robust Regression: rlm.

Normilization: Normilization scales data to a common range usually between 0 & 1.

Types of normilization: 1.minimum and maximum, 2.Standardisation.

User defined function: A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.

1.Function declaration.

2.Function definition.

3.Function calling.

Syntax:

```
Function_name<-function(args n){  
    variable<-operations on args n  
    return(variable)  
}
```

Ex:

```
add_numbers<-function(x,y){  
    return(x+y)  
}
```

```
result<-add_numbers(5,5)
```

Function on function objects: In R, function are first class citizens, meaning they can be passed as arguments to other function, returned from function and stored in variable.

Passing a function as arguments:

Ex:

```
apply_function <- function(func, x) {  
    func(x)  
}
```

```
square <- function(x) {  
    x * x  
}
```

```
result <- apply_function(square, 5)  
print(result)
```

Scope of variables: The scope of a variable refers to the region of code where the variable is accessible.

Types of Variables:

- 1.Global scope: Variable defined outside the loop/function.
- 2.Local scope: Variable defined inside the loop/function.
- 3.Parent scope: Scope of the enclosing environment.

Ex:

```
x<-10  
a<-function(){  
    x<-20  
    print(x)  
    x<-30  
}
```

```
a()
```

```
print(x)
```



NOTE: In R, global variable can be accessed using "<-<" as a keyword.

Environment closures: A closure is a function that has access to its own scope/space and a scope of its enclosing environment.

Ex:

```
outer<-function(){  
  x<-10  
  inner<-function(){  
    print(x)  
  }  
  return(inner)  
}  
f<-outer()  
f()
```

Recursion: Recursion is an programming technique where a function calls itself again and again.

Ex:

```
factorial<-function(n){  
  if(n==0){  
    return(1)  
  }  
  else{  
    return(n*factorial(n-1))  
  }  
}  
print(factorial(5))
```

Lexical scoping: R uses lexical scoping which means that the scope of the variable is determined by its position in the code.

Ex:

```
x<-10
```

```
f<-function(){
```

```
  y<-20
```

```
  g<-function(x){
```

```
    print(x)
```

```
    print(y)
```

```
  }
```

```
  g(x)
```

```
}
```

```
f()
```

Dynamic scoping: This scope is determined at the run time.

Variable look ups: R follows the following order to look ups the variable.

- 1.Local scope.
- 2.Parent scope.
- 3.Global scope.
- 4.Built-in function.

Variable making: It occurs when global variable and local variable both have the same name.

Ex:

```
x<-10
```

```
f<-function(){
```

```
  x<-20
```

```
  print(x)
```

```
}
```

```
f()
```

```
print(x)
```

Function as closure: Function can be used as the closure to encapsulate data and behaviour.

Ex:

```
counter<-function(){  
  x<-0  
  i<-function(){  
    x<-x+1  
    return(x)  
  }  
  return(i)  
}  
c<-counter  
print(c())
```